Universidad Distrital Francisco José de Caldas

Faculty of Engineering
Computer Engineering Program

Workshop 2 – Advanced Learning Models (Part A)

Leakage-Aware MLP Regression for Ames Housing: A Controlled Comparison with Classical Baselines

Group 1 Authors

Miguel Alejandro Chávez Porras – 20212020033
Luisa Fernanda Guerrero Ordoñez – 20212020099
Dilan Stive Arboleda Zambrano – 20212020105

Machine Learning – Semester 2026-III
Bogotá, Colombia

2026

Contents

1 Introduction **1**

2 Methods and Materials **2**

2.1 Dataset and Leakage-Aware Preprocessing 2

2.2 Network and Training Protocol 2

3 Experimental Setup **3**

3.1 Initial Failure and Conditioning Correction 3

3.2 Controlled Comparison, Ablations, and Selection 4

4 Results and Discussion **4**

4.1 Optimization, Ablation, and Stability Results 4

4.2 Final Test Comparison 4

4.3 Predictions, Residuals, and Extreme Cases 6

5 Reproducibility and Limitations **7**

6 Conclusions **7**

Abstract

Residential price prediction from heterogeneous tabular data requires nonlinear modeling while remaining robust to skewed targets, sparse categories, and rare high-value transactions. This work develops a leakage-aware multilayer perceptron (MLP) for the Ames Housing dataset, diagnoses an initial optimization failure, and evaluates controlled changes in architecture, dropout, weight decay, learning rate, and train-only standardization. The selected 261-128-64-1 network achieved a test mean absolute error (MAE) of \$14,666.23 on 440 held-out properties, improving the Workshop 1 Ridge, Random Forest, and Gradient Boosting baselines by 2.43%, 12.18%, and 1.42%, respectively. Random Forest nevertheless retained the best root mean squared error and coefficient of determination, while Gradient Boosting retained the best logarithmic error. The results show that target conditioning and stable validation procedures can make an MLP competitive on a small tabular dataset, but do not establish universal neural-network superiority.

Keywords: Ames Housing, multilayer perceptron, regression, tabular data, reproducibility, model comparison.

1 Introduction

Automated residential valuation supports taxation, lending, investment, and market analysis, but accurate estimation is difficult because price depends on heterogeneous physical, temporal, quality, amenity, and location attributes. Ames Housing was introduced by De Cock as a modern alternative to the Boston Housing dataset and contains 2,930 sales recorded in Ames, Iowa, from 2006 to 2010, with 80 explanatory variables spanning nominal, ordinal, discrete, and continuous measurements [1]. Structural missingness, rare categories, nonlinear interactions, and a right-skewed price distribution make it a useful supervised-regression benchmark.

Workshop 1 established three classical baselines. Ridge Regression controls linear-model variance through coefficient shrinkage; Random Forest averages randomized trees [2]; and Gradient Boosting constructs an additive predictor through stagewise functional optimization [3]. All were implemented with scikit-learn [4] and evaluated on a fixed split. They remain strong references because tabular benchmarks show that tree ensembles frequently match or outperform neural networks on small and medium datasets [5, 6].

An MLP is nevertheless an appropriate complementary model because successive affine transformations and nonlinear activations can approximate complex multivariate functions [7]. Rectified linear units (ReLU) provide a simple non-saturating activation [8], while Kaiming initialization stabilizes signal propagation through rectifier networks [9]. With limited observations, flexibility must be controlled: dropout discourages hidden-unit co-adaptation [10], Adam supplies adaptive first- and second-moment optimization [11], AdamW decouples weight decay from the adaptive update [12], and validation-based early stopping limits unnecessary fitting [13]. The implementation uses PyTorch [14].

This study asks whether a carefully conditioned MLP can improve the Workshop 1 baselines under the same observations and metric definitions. Its contributions are: (i) diagnosis of a failed initial run; (ii) a controlled architecture, regularization, learning-rate, and standardization comparison; (iii) ablations of input and target standardization; (iv) multi-seed stability analysis; and (v) one final evaluation of a checkpoint frozen using validation only. The goal is a reproducible, metric-aware comparison rather than a claim that deep learning is intrinsically superior.

2 Methods and Materials

2.1 Dataset and Leakage-Aware Preprocessing

The raw table contains 2,930 observations, 80 predictors, and the target `SalePrice`. Prices range from \$12,789 to \$755,000, with median \$160,000 and mean \$180,796. Twenty-seven columns contain missing values; many denote the physical absence of a pool, garage, basement, fireplace, alley, or fence rather than an unknown measurement.

The immutable `data/splits.json` assigns 2,050 records to training and 440 each to validation and test, approximately 70/15/15. The Workshop 1 preprocessor is fitted on training only. Two atypical training transactions with above-ground living area over 4,000 square feet and price below \$300,000 are removed, leaving 2,048 effective training examples; validation and test remain unfiltered. Structural absence is encoded as "None" or zero, lot frontage is imputed with neighborhood medians learned from training, other missing values use training modes or medians, ordered categories receive monotonic encodings, nominal variables are one-hot encoded with unknown categories ignored, and numerical variables are robustly scaled.

Feature engineering adds total usable area, weighted bathrooms, house and remodeling ages, porch area, amenity indicators, and a quality-condition interaction. The resulting matrix has 261 inputs. A diagnostic revealed that robust scaling alone left 741 training values with magnitude above 100 because several near-zero-interquartile-range variables retained large scales; `Misc Val` reached 15,500. Consequently, a second mean-standard-deviation transform was fitted to the processed training matrix and applied unchanged to validation and test.

The target is first mapped to $y_{\log} = \log(1+y)$ and then standardized with training statistics:

$$z_y = \frac{y_{\log} - \mu_{\text{train}}}{\sigma_{\text{train}}}, \tag{1}$$

where $\mu_{\text{train}} = 12.013993$ and $\sigma_{\text{train}} = 0.395353$. Inference returns predictions to dollars using

$$\hat{y} = \exp(\hat{z}_y \sigma_{\text{train}} + \mu_{\text{train}}) - 1. \tag{2}$$

Every imputation, encoding, and scaling statistic is therefore learned without validation or test information.

2.2 Network and Training Protocol

Figure 1 summarizes the complete workflow and selected architecture. The final MLP maps 261 inputs through hidden layers of 128 and 64 ReLU units. Dropout with probability 0.05 follows each hidden activation, and a final linear unit predicts the standardized log target. The model contains 41,857 trainable parameters. Hidden weights use Kaiming normal initialization, the output weights use Xavier uniform initialization, and all biases start at zero.

Training minimizes MSE on z_y. AdamW uses learning rate 10^{-3} and weight decay 10^{-5}. Mini-batches contain 32 observations; gradient norm is clipped at 5. A ReduceLROnPlateau scheduler halves the learning rate after eight validation-loss plateaus, with minimum 10^{-5}. Runs are allowed 250 epochs. Early stopping monitors validation MAE in dollars with patience 30, while the scheduler monitors validation MSE on the model scale. Seeds control Python, NumPy, PyTorch, and mini-batch order. The saved checkpoint contains model weights, architecture, feature order, split metadata, and both training-only standardizers.

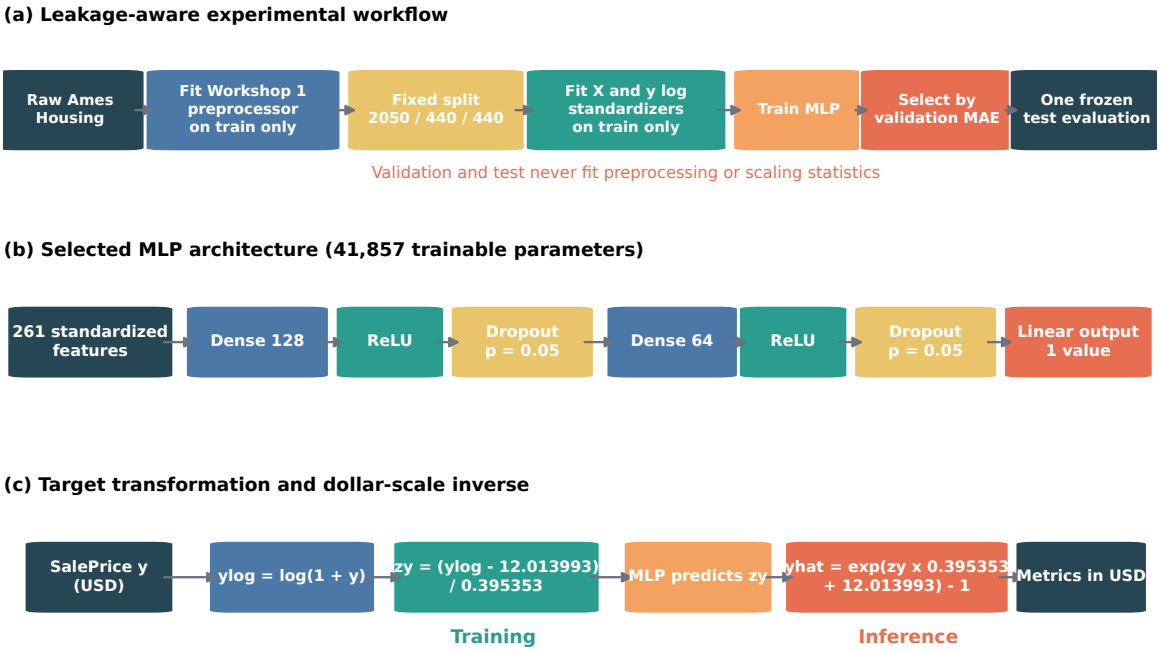


Figure 1: Leakage-aware methodology. (a) Data operations and selection order; (b) exact selected architecture, including dropout after both hidden ReLU layers; and (c) training-target transformation and inverse mapping used before dollar-scale metrics.

3 Experimental Setup

3.1 Initial Failure and Conditioning Correction

The initial network used hidden widths 128-64-32, dropout 0.20, Adam, learning rate 10^{-3}, and 43,905 parameters. It optimized MSE directly on $\log(1+\text{SalePrice})$ without additional input or target standardization. Validation loss oscillated between 1.093 and 16.563, early stopping occurred after 36 epochs, and the best MAE was \$77,990.05 with negative R^2. This behavior is consistent with an output initialized near zero being asked to approximate an uncentered target near 12 while large feature magnitudes dominate early gradients.

Experiment 2 introduced train-only standardization of inputs and target, reduced the network to 64-32, lowered dropout to 0.05, changed Adam to AdamW, and added matched initialization, scheduling, and clipping. Table 1 shows the resulting 79.69% MAE reduction. Because multiple changes were applied together, subsequent controlled experiments isolated their effects where possible.

Table 1: Initial Diagnosis and Corrected Reference

Validation result	Initial MLP	Experiment 2
Hidden units	128-64-32	64-32
Parameters	43,905	18,881
Best epoch	16	75
MAE (USD)	77,990.05	15,837.95
RMSE (USD)	174,526.51	24,233.63
R^2	-2.9268	0.9243

3.2 Controlled Comparison, Ablations, and Selection

All seed-42 comparisons reuse the same 2,048 effective training rows and 440 validation rows. The experiment-2 reference uses 64-32 hidden units, dropout 0.05, weight decay 10^{-5}, and learning rate 10^{-3}. Four runs change one factor: dropout to 0, architecture to 128-64, weight decay to 10^{-4}, or learning rate to 5×10^{-4}. Two ablations remove the additional input standardization or target standardization.

Table 2: Controlled Validation Experiments with Seed 42

ID	Hidden units	Changed factor	Standardization	Parameters	Epoch	MAE (USD)	R^2
Reference	64-32	None	Inputs and target	18,881	75	15,837.95	0.9243
E3	64-32	Dropout 0.00	Inputs and target	18,881	66	19,330.28	0.8921
E4	128-64	Architecture	Inputs and target	41,857	83	**14,051.46**	**0.9452**
E5	64-32	Weight decay 10^{-4}	Inputs and target	18,881	81	15,534.41	0.9280
E6	64-32	Learning rate 5×10^{-4}	Inputs and target	18,881	61	16,341.55	0.9188
A1	64-32	No extra input scaling	Target only	18,881	75	15,696.59	0.9108
A2	64-32	No target scaling	Inputs only	18,881	152	34,600.95	0.5847

The reference and the two strongest non-ablation alternatives were repeated with seeds 7, 42, and 2026. Configuration selection used the lowest mean validation MAE across the three seeds. The 128-64 model achieved $14,238.79 \pm 183.95$, compared with $14,379.15 \pm 1,037.55$ for weight decay 10^{-4} and $14,695.31 \pm 1,021.94$ for the reference. Although stronger weight decay produced the best single run ($13,526.67 at seed 7), its variability made it a less reliable choice. Among the three already-selected 128-64 runs, seed 42 had the lowest validation MAE ($14,051.46) and was frozen before test evaluation.

4 Results and Discussion

4.1 Optimization, Ablation, and Stability Results

Figure 2(a) shows that target and input conditioning eliminated the extreme validation-MAE trajectory of the initial run. In the controlled study, removing target standardization was the most damaging ablation: MAE increased to $34,600.95 and the best epoch moved to 152. The output layer otherwise begins near zero while the target remains near 12, producing unnecessarily large initial errors. Removing additional input standardization increased first-epoch MAE from $31,590.70 to $74,448.73 and worsened final RMSE and R^2, although its single-seed minimum MAE remained close to the reference. Thus, target standardization is directly supported by the ablation, while input standardization is supported mainly as an optimization and tail-error safeguard.

Removing dropout worsened MAE by $3,492.33, consistent with overfitting. Increasing weight decay improved the seed-42 reference by $303.53 but was unstable across seeds. Lowering the initial learning rate worsened MAE by $503.60. Expanding the network to 128-64 produced the most consistent gain: under seed 42 it reduced MAE by $1,786.48 (11.28%) relative to experiment 2, and across seeds it had the lowest mean and standard deviation. The learning curves in Fig. 3 also show a persistent train-validation gap, so the selected checkpoint is determined by validation MAE rather than final training loss.

4.2 Final Test Comparison

The frozen seed-42 checkpoint was loaded without retraining and evaluated once on the 440 fixed test properties. Input normalization uses checkpoint statistics, and output inversion precedes all metrics.

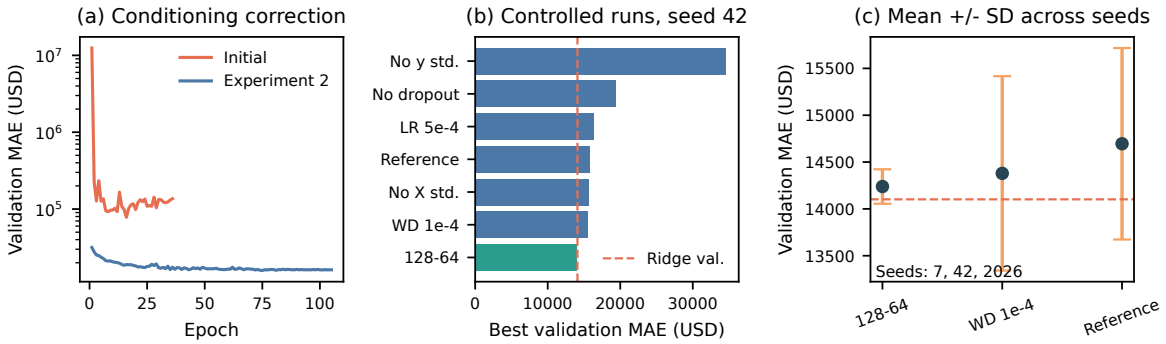


Figure 2: Optimization and model-selection evidence. (a) Validation MAE before and after conditioning; (b) controlled seed-42 comparison, including the Workshop 1 Ridge validation reference; and (c) mean $\pm$ standard deviation over seeds 7, 42, and 2026.

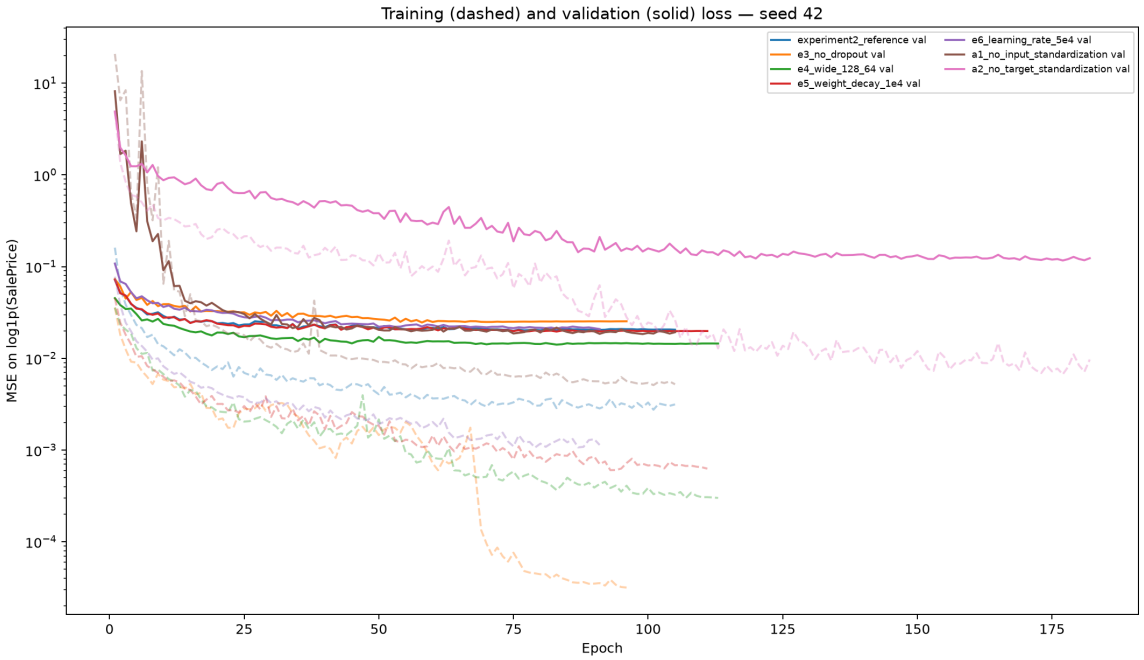


Figure 3: Training (dashed) and validation (solid) MSE for the controlled seed-42 experiments. The logarithmic vertical scale exposes the failed no-target-standardization ablation and the train-validation separation.

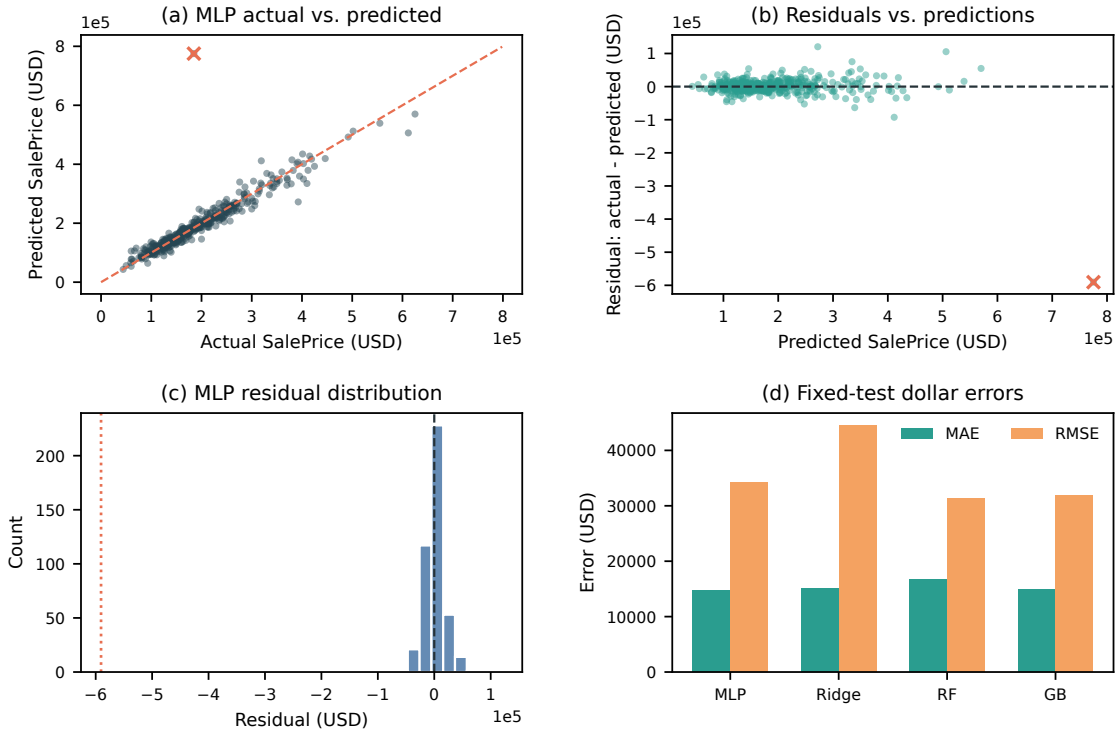


Figure 4: Fixed-test diagnostics. (a) MLP actual versus predicted prices; (b) residuals versus predicted price; (c) residual distribution, with the extreme error marked; and (d) dollar-scale comparison with Workshop 1 baselines. Red crosses or dotted lines identify the largest absolute MLP residual.

Table 3 uses the immutable Workshop 1 test metrics and the new MLP metrics under the same definitions: MAE and RMSE in dollars, dollar-scale R^2, and RMSE on $\log(1+y)$.

Table 3: Definitive Performance on the Fixed Test Partition ($N = 440$)

Model	MAE (USD)	RMSE (USD)	R^2	$\text{RMSE}_{\log}$
Deep Learning MLP	**14,666.23**	34,133.71	0.8483	0.1254
Ridge Regression	15,031.51	44,511.47	0.7421	0.1254
Random Forest	16,700.76	**31,325.66**	**0.8723**	0.1325
Gradient Boosting	14,876.80	31,939.29	0.8672	**0.1208**

The MLP achieved the smallest MAE, improving Ridge by \$365.28 (2.43%), Random Forest by \$2,034.53 (12.18%), and Gradient Boosting by \$210.57 (1.42%). Its test MAE was 4.38% above its validation MAE. The ranking changes with the metric: Random Forest obtained the best RMSE and R^2, while Gradient Boosting obtained the best logarithmic error. Therefore, the MLP improved typical absolute dollar deviation on this split but did not dominate squared-error or relative-error behavior.

4.3 Predictions, Residuals, and Extreme Cases

Figure 4(a) shows that most MLP predictions follow the identity line through the common price range. Residuals are concentrated near zero, with mean \$217, but their spread increases for expensive predictions. The largest error occurs at split index 2181 (PID 908154205): the actual price is \$184,750 and the MLP predicts \$775,256.31, producing residual $-\$590,506.31$. This unusual high-area, low-price property was retained in test by design and was not used to revise the model.

Because RMSE squares every residual before averaging, this single error contributes far more than a typical observation. It substantially increases MLP RMSE and lowers R^2, explaining how the network can lead in MAE while trailing both ensembles under tail-sensitive criteria. The diagnostic plots use reconstructed baseline predictions, whereas Table 3 retains the historical Workshop 1 metrics.

5 Reproducibility and Limitations

The checkpoint was loaded into two independent model instances and produced identical prediction arrays. Seven final-evaluation tests verified validation-only selection, architecture metadata, training-only scaler statistics, the 440 ordered test indices, inverse target transformation, saved metric reproduction, and prediction-file identity. Three unit tests separately verified standardizer behavior, target round-trip accuracy, and model output shape. All ten tests passed, and the report notebook executes without retraining while rendering saved tables and figures.

The Workshop 1 baselines were reconstructed with frozen hyperparameters for diagnostic plots. In the current software environment their MAE values differed from the historical JSON by \$0.72 for Ridge, \$14.75 for Random Forest, and \$145.34 for Gradient Boosting. These small differences are consistent with numerical or library-version variation. To avoid presenting approximate reproduction as exact, the definitive table uses the immutable Workshop 1 metrics while the diagnostic plots use current predictions.

The evaluation has important limits. Only 2,048 effective training examples support 41,857 parameters. A single fixed split means seed repetitions measure initialization and mini-batch variability, not variation across data partitions. Only three configurations received multi-seed evaluation, so the search is controlled but not exhaustive. Test MAE across the three previously selected 128-64 checkpoints ranged from \$14,666.23 to \$16,746.04, with descriptive mean \$15,735.87 $\pm$ \$1,041.18; these values were not used to replace seed 42 or form an ensemble. Finally, the test set has now been observed and must not support further model selection.

6 Conclusions

A two-hidden-layer MLP can be competitive on Ames Housing when the processed inputs and log-transformed target are standardized using training statistics only. The initial failure demonstrates that a reasonable architecture and optimizer do not compensate for poor conditioning. Controlled experiments identified target standardization as the most important correction, light dropout as useful regularization, and 128-64 as the most stable evaluated architecture. Validation-only selection and checkpoint metadata preserved separation from the fixed test partition.

The frozen seed-42 model achieved the best test MAE among the four compared models at \$14,666.23. The outcome is nevertheless metric-dependent: Random Forest and Gradient Boosting were more robust under squared-error criteria, and one extreme property exposed a substantial MLP failure. Future work should evaluate robust losses, uncertainty estimates, and tabular-specific architectures under a new validation protocol rather than reuse the now-observed test set.

References

[1] D. De Cock, "Ames, iowa: Alternative to the boston housing data as an end of semester regression project," *Journal of Statistics Education*, vol. 19, no. 3, 2011.

[2] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.

[3] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[4] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[5] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, "Revisiting deep learning models for tabular data," in *Advances in Neural Information Processing Systems 34*, 2021.

[6] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in Neural Information Processing Systems 35*, 2022.

[7] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989.

[8] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th International Conference on Machine Learning*, 2010, pp. 807–814.

[9] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification," in *Proceedings of the IEEE International Conference on Computer Vision*, 2015, pp. 1026–1034.

[10] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[11] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations*, 2015.

[12] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019.

[13] L. Prechelt, "Early stopping—but when?" in *Neural Networks: Tricks of the Trade*, ser. Lecture Notes in Computer Science, G. B. Orr and K.-R. Müller, Eds. Springer, 1998, vol. 1524, pp. 55–69.

[14] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, 2019, pp. 8024–8035.